РЕШЕН ИЗПИТ 5 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 5: Фитнес — Месечни абонаменти | 100 точки

Теми: typedef struct + realloc шаблон | средна стойност + филтриране | бинарно търсене по ID

НАЙ-ПРЕДСТАВИТЕЛНИЯТ ИЗПИТ — всичките три задачи са класически шаблони.

Задача 3 носи 40 точки — бинарното търсене е приоритет номер едно.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА
`статичен низ, дължина 55`	char names[55]; — НЕ char*, никога char* в struct!
`формат "AA1234" (6 символа)`	char id[7]; — 6 символа + 1 за нулевия терминатор
`реално число, формат 0.00`	float price; — %.2f при printf
`цяло трицифрено число`	int locker; — обикновен int
`динамично разширява масива`	realloc() шаблон в addMember()
`записва в txt с разделител ';'`	fprintf с fopen("a") — добавяне, не презаписване

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

> **ВАЖНО:** Прочети целия код преди да четеш обясненията. Опитай да предположиш какво прави всяка функция.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char  names[55]; /* до 54 символа + '\0' — НИКОГА char* */
    char  id[7];     /* AA1234 = 6 символа + '\0' */
    float price;     /* реално число */
    int   locker;    /* цяло число */
} Member;

/* ── Задача 1: addMember() ── */
Member* addMember(Member *arr, int *count) {
    (*count)++;
    arr = realloc(arr, (*count) * sizeof(Member));
    if (!arr) { printf("Error\n"); exit(1); }
    Member *m = &arr[(*count) - 1];
    printf("Names: "); scanf("%54s", m->names);
    printf("ID: ");    scanf("%6s",  m->id);
    printf("Price: "); scanf("%f",  &m->price);
    printf("Locker: "); scanf("%d", &m->locker);
    FILE *f = fopen("membersText.txt", "a");
    if (!f) { printf("Error\n"); return arr; }
    fprintf(f, "%s;%s;%.2f;%d\n",
            m->names, m->id, m->price, m->locker);
    fclose(f);
    return arr;
}

/* ── Задача 2: filterBelowAverage() ── */
void filterBelowAverage(Member *arr, int count) {
    if (count == 0) return;
    float sum = 0;
    for (int i = 0; i < count; i++) sum += arr[i].price;
    float avg = sum / count;
    for (int i = 0; i < count; i++)
        if (arr[i].price < avg)
            printf("%s - %s - %.2f\n",
                   arr[i].names, arr[i].id, arr[i].price);
}

/* ── Задача 3: saveBinary() + searchBinary() ── */
void saveBinary(Member *arr, int count) {
    FILE *f = fopen("members.bin", "wb");
    if (!f) { printf("Error\n"); return; }
    fwrite(arr, sizeof(Member), count, f);
    fclose(f);
}

void searchBinary(char *searchId) {
    FILE *f = fopen("members.bin", "rb");
    if (!f) { printf("Error\n"); return; }
    Member m;
```

```c
    int found = 0;
    while (fread(&m, sizeof(Member), 1, f) == 1) {
        if (strcmp(m.id, searchId) == 0) {
            printf("Bin Names: %s\n",    m.names);
            printf("Bin ID: %s\n",       m.id);
            printf("Bin M_Price: %.2f\n", m.price);
            printf("Bin Locker: %d\n",   m.locker);
            found = 1; break;
        }
    }
    if (!found) printf("Не е намерен.\n");
    fclose(f);
}

int main() {
    Member *arr = NULL;
    int count = 0;
    arr = addMember(arr, &count);
    arr = addMember(arr, &count);
    arr = addMember(arr, &count);
    filterBelowAverage(arr, count);
    saveBinary(arr, count);
    char id[7];
    printf("ID: "); scanf("%6s", id);
    searchBinary(id);
    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — ЗАДАЧА 1: СТРУКТУРАТА И addMember()

Ред по ред: декларацията

КОД	ОБЯСНЕНИЕ
`typedef struct {`	Дефинираме нов тип. typedef = после пишем само Member, не struct Member.
`    char names[55];`	55 = 54 символа + 1 за '\0'. НИКОГА char *names — при fwrite/fread char* записва само адреса!
`    char id[7];`	AA1234 = 6 символа, затова 6+1=7. Формула: char field[макс_символа + 1].
`    float price;`	Задачата казва 'формат 0.00' — при printf ползваме %.2f.
`    int locker;`	Просто int — няма специален тип за трицифрено.
`} Member;`	Затваряме блока и даваме името Member. От тук: Member m; Member *arr;

```
[NE]  char *names;       // само адрес (8 байта) — fwrite ще запише адреса, не текста!
[OK]  char names[55];    // 55 байта ДИРЕКТНО в структурата — fwrite работи
```

Ред по ред: addMember()

КОД	ОБЯСНЕНИЕ
`Member* addMember(`	Връща Member* — ЗАДЪЛЖИТЕЛНО! realloc може да премести масива.
`    Member *arr, int *count)`	count е указател — за да модифицираме брояча в main(). Ако беше int count, промените щяха да се губят.
`    (*count)++;`	(*count) разкожва указателя. Без скоби: *count++ увеличава адреса — bug!
`    arr = realloc(arr, ...)`	ЗАДЪЛЖИТЕЛНО arr = realloc. Без arr = губим новия адрес.
`    if (!arr) { exit(1); }`	realloc връща NULL ако паметта е свършила. Тази проверка носи точки!
`    Member *m = &arr[(*count)-1];`	Адрес на ПОСЛЕДНИЯ елемент. По-удобно от arr[(*count)-1].names.
`    scanf("%54s", m->names);`	БЕЗ & пред m->names — масивът вече е адрес! %54s = максимум 54 символа.
`    scanf("%f", &m->price);`	ЗА числа задължително &. m->price е стойност, не адрес.

КОД	ОБЯСНЕНИЕ
`fopen("membersText.txt", "a")`	"a" = append. Всяко добавяне добавя нов ред. "w" би изтривало файла!
`fclose(f);`	ЗАДЪЛЖИТЕЛНО. Без fclose() данните може да са в буфер.
`return arr;`	ЗАДЪЛЖИТЕЛНО! В main(): arr = addMember(arr, &count);

СЕКЦИЯ 3 — ЗАДАЧА 2 И ЗАДАЧА 3

Задача 2: два цикъла — защо не един?

Не можем да знаем средната стойност, докато не обходим ВСИЧКИ елементи. Затова: първи цикъл = изчисляване на avg, втори цикъл = принтиране под avg.

КОД	ОБЯСНЕНИЕ
`if (count == 0) return;`	Деление на нула при sum/count = undefined behavior. Винаги проверявай!
`float sum = 0;`	Задължителна инициализация с нула. Без нея — случайна стойност.
`float avg = sum / count;`	sum е float → резултатът е с плаваща запетая.
`if (arr[i].price < avg)`	СТРОГО по-малко (<). Прочитай условието — понякога искат <= или >.

Задача 3: ред по ред

КОД	ОБЯСНЕНИЕ
`fopen("members.bin", "wb")`	'b' = binary ЗАДЪЛЖИТЕЛНО. Без 'b' на Windows данните се разрушават!
`fwrite(arr, sizeof(Member), count, f)`	Записва count * sizeof(Member) байта директно. arr (без &) е адрес на първия елемент.
`fopen("members.bin", "rb")`	'rb' задължително и при четене.
`while (fread(&m, sizeof(Member), 1, f) == 1)`	fread връща броя прочетени елементи. == 1 = успех. При EOF → 0 → излизаме.
`if (strcmp(m.id, searchId) == 0)`	strcmp сравнява СЪДЪРЖАНИЕТО. НИКОГА m.id == searchId — сравнява АДРЕСИ!
`found = 1; break;`	Намерихме — флаг нагоре и излизаме. Без break четем целия файл напразно.

ТОП ГРЕШКИ

```
[NE]  realloc(arr, n*sizeof(Member))     // arr не се обновява!
[OK]  arr = realloc(arr, n*sizeof(Member)) // задължителното присвояване
```

```
[NE]  void addMember() без return arr   // caller не вижда новия адрес
[OK]  Member* addMember() { return arr; } // тип и return са задължителни
```

```
[NE]  fopen("data.bin", "r")            // "r" за текстов — данните се
разрушават
[OK]  fopen("data.bin", "rb")           // "b" е задължително

[NE]  if (m.id == searchId)             // сравнява АДРЕСИ
[OK]  if (strcmp(m.id, searchId) == 0)  // сравнява СЪДЪРЖАНИЕТО
```